LiTS on BlocksWorld

A partial reproduction and a bounded cycle-pruning study

Individual reproducibility report | Agentic AI for Business and FinTech

Abstract

This study runs the authors' LiTS repository on a small BlocksWorld subset using DeepSeek Flash. It reproduces the runnable multi-step planning and offline-evaluation workflow, rather than claiming an exact reproduction of the paper's numerical result. An engineering-fixed MCTS control is compared with one isolated modification: pruning a modeled state that repeats an ancestor on the same path. The same 20 tasks are run three times per arm under identical per-task limits. Control and pruning achieve 24/60 and 25/60 successful task-runs, respectively. Pruning reduces recorded API calls by 34.7% and estimated charges by 34.1%, but accuracy differences change sign across rounds. Offline inspection of three round-three losses finds genuine cycles, independently sampled action differences, single-candidate bottlenecks, and positive value backup from a no-progress loop. A later two-task low-score-widening pilot activates correctly but produces no success gain. The defensible finding is a repeatable reduction in resource use on this subset, not a reliable accuracy improvement.

Study at a glance

Item	Scope
Software	LiTS 0.4.0; pinned upstream commit 1ef347bee7b3f2c10dc504342589b8c951794c2d
Primary experiment	20 fixed six-step-source tasks; 3 rounds; 2 arms; 120 task-runs
Primary change	Path-local ancestor-cycle pruning on a shared engineering-fixed control
Observed outcome	Fewer calls in all rounds; net success differences +1, +2, -2
Exploratory addition	Low-score singleton widening; only 2 development tasks per arm

Research questions

RQ1: Can the repository produce legal, goal-reaching plans and reusable run artifacts under an accessible replacement model?
RQ2: Does ancestor-cycle pruning reduce resource use under a fixed search-attempt limit, and what happens to success?
RQ3: What do the failure logs reveal, and does a bounded candidate-widening pilot address the observed bottleneck?

The accompanying evidence package contains source/configuration fingerprints, task IDs, per-request usage, saved trees, independent replay results and offline reporting commands. No new paid experiment was run while preparing this report.

Name: WEI ZIJUN Student ID: 1155244573

1. Target claim and scope of reproduction

LiTS separates planning into Policy, Transition and RewardModel components and supports registered search extensions [1,2]. This report targets its runnable BlocksWorld workflow and external-extension behavior. Published benchmark numbers provide context, not a directly matched baseline.

Published reference [1]	This project	Interpretation
Table 3: MCTS accuracy 66.7% on 30 examples	Engineering control: 40.0% average over 20 tasks repeated 3 times	Not a direct replication: model, sample identities and search budget differ.
Claude 3.5 Sonnet via AWS Bedrock	DeepSeek Flash via the official OpenAI-compatible endpoint	Underlying model substitution is a major controlled departure.
10 MCTS iterations; branch 3; depth 6; stop at first solution	3 iterations; branch 3; depth 6; stop at first solution	Three iterations are a project budget setting, not a reported paper ablation.
488K output tokens; US$21.99 for the published MCTS run	48,623 output tokens and CNY 1.0003-2.0007 estimated for 60 control task-runs	Different run counts, providers and currencies; no cross-paper cost-saving claim.

The reference settings above are from Section 3.1 and Table 3, not an assertion that the published 30 tasks match the bundled source file. The exact paper task IDs were not recovered. The pinned repository snapshot is dated 7 October 2026, after paper v2 (17 May 2026), so it is not guaranteed to be the paper-time implementation. The paper's Chain result is not reproduced here. Continuation/BNE is disabled; no BNE effect is claimed.

What was actually reproduced?

Real API runs exercised the authors' task loader, environment policy and reward interfaces, LLM-generated state transitions, MCTS infrastructure, checkpoints and terminal-node serialization. Independently valid six-action plans were observed. External subclasses were loaded through the repository's registry without editing tracked upstream files. These observations support a **partial workflow reproduction under replacement-model conditions**. They do not recover the published 66.7% result.

Why the control is engineering-fixed

Initial repository runs exposed duplicate finite actions and repeated selection of exhausted branches. Shared fixes were introduced before the primary comparison: enumerate remaining legal actions, retain distinct existing siblings, skip exhausted frontiers, and widen rollout-only nodes when selected. The primary control is therefore **UniquePolicy + FrontierMCTS**, not untouched author MCTS. Both primary arms use these fixes; the isolated treatment adds only ancestor-cycle pruning.

Earlier 10-versus-3 pilot runs, whole-plan prompting and local symbolic search were debugging explorations. Their results are not pooled with the primary experiment. In particular, a symbolic solver's success is not counted as a LiTS/LLM success.

2. Environment, tasks and evaluation protocol

Setting	Recorded value
Runtime	macOS 26.6 arm64; Python 3.12.14; NumPy 2.2.6; PyTorch 2.6.0; OpenAI SDK 1.78.1
Model / endpoint	openai/deepseek-flash; https://api.deepseek.com; thinking explicitly disabled
Policy decoding	Temperature 0.7 (1e-6 at the depth boundary); top-p 0.95
Reward / transition decoding	PRM temperature 0.6, top-p 1.0, normally 10 output tokens/request; transition 1e-6, top-p 1.0
Search	3 iterations; branching 3; max depth 6; rollout limit 6; rollout candidates 1; confidence samples 1
Task limits	150 API starts; 150,000 cumulative returned tokens checked before each next call; 512 output-token ceiling; 900-second process timeout
Seeds and scheduling	Local Python/NumPy/PyTorch seed 42 in every round; no provider seed; 4 concurrent isolated task processes

Decoding values are reconstructed from the pinned call paths and configs, not a complete HTTP-parameter capture. The 512 limit is a ceiling: binary PRM requests normally ask for fewer tokens and may retry up to three times to obtain a recognized label. The response model is recorded only as an alias; the provider weight snapshot is unknown. A last response can cross the returned-token threshold because the guard checks cumulative usage before the next request.

Task selection and freeze

The bundled `bw_data_step_6.json` contains 114 records. The first two in upstream order form development tasks; the next 20 form the fixed test subset. These are consecutive, not randomly sampled, and are not claimed to be the paper's task IDs. SHA256 manifests pin the data, PDDL inputs and upstream commit. The primary protocol was frozen after the development comparison and before the first 20-task test. Two later repetitions kept code, task order, local seed and limits unchanged. Each task starts in a fresh process with a fresh tree. Prompts may benefit from provider-side input caching.

Success and resource definitions

Primary success requires replaying the selected terminal plan from the frozen initial state under independent pick-up, put-down, unstack and stack rules, then satisfying every goal relation. Official final-text goal checks are reported separately. A depth-limited leaf is not a solution. Incomplete or budget-interrupted tasks remain failures in the planned denominator.

Calls count completed, usage-bearing API requests; started-but-unaccounted requests are tracked separately. **Returned tokens** mean input plus output, including cached input. Fees are calculated from actual cache-hit, cache-miss and output counts using the 4 November price snapshot [4]: CNY per million tokens is 0.02/1/4 off-peak and 0.04/2/8 peak, respectively. The interval is not a checked provider bill.

Each arm has 60 repeated task-run observations but only 20 unique test tasks. Standard deviations below are sample SDs across three round-level values, not confidence intervals from 60 independent samples.

3. Isolated modification and validation

Path-local cycle pruning

CycleMCTS inherits the shared FrontierMCTS control. After an LLM transition, state text is parsed into block-to-support positions. Position completeness, hand status, clear predicates and stack structure must agree. A canonical state equal to any ancestor on the same path is marked closed to further expansion. The child and its action remain in the tree for audit and duplicate exclusion. A cycle is never labeled as a goal.

```
if complete_consistent_state(current):
    if canonical(current) in canonical(ancestors):
        mark_cycle_closed(current)
        stop_expanding_this_branch()
# Invalid/incomplete text: retain branch (fail open).
# No global transposition merge, no symbolic transition.
```

This uses only already generated model states. It does not look ahead with a symbolic planner, read reference plans, replace LLM dynamics or change reward backup. A separate audit records each closed node, its matching ancestor, depth and canonical positions.

Controlled comparison

Shared in both arms	Only treatment difference
Same task IDs, model, prompt templates and decoding paths	mcts_frontier -> mcts_cycle
Same 3 attempts, depth, branching and usage caps	A recognized ancestor revisit is closed.
Same candidate choice, reward model and backup	No extra recovery attempt is granted after a cycle.
Same independent evaluator and request ledger	Cycle audit metadata is additionally written.

Important interpretation: the arms have the same maximum limits, not equal realized computation. A closed loop can finish a rollout early while still consuming one of the three outer iterations. Thus fewer calls can reflect earlier stopping rather than more efficient use of an equal amount of search. This stopping change is part of the measured treatment.

Offline checks before paid runs

Network-disabled regressions cover reordered state text, conflicting/missing blocks, hand/clear inconsistencies, invalid stack structures, ancestor-only matching, preservation of real goals, unknown states becoming known, and a cycle discovered at the final rollout/depth boundary. Budget exceptions must propagate while preserving partial trees and cycle audits. Additional checks verify finite-action uniqueness, API output ceilings, disabled SDK retries and independent evaluator rejection of illegal plans.

Development on two tasks yielded 1/2 successes in each arm, with calls 97 to 67. Code/config hashes were then frozen. The three main tests completed all 120 planned task-runs, with zero budget interruptions and zero unaccounted requests.

4. Main results: three repeated test rounds

Round	Control success	Pruned success	Control calls	Pruned calls	Call reduction	Net success
1	7/20	8/20	1137	721	36.6%	+1
2	7/20	9/20	1147	718	37.4%	+2
3	10/20	8/20	1049	737	29.7%	-2

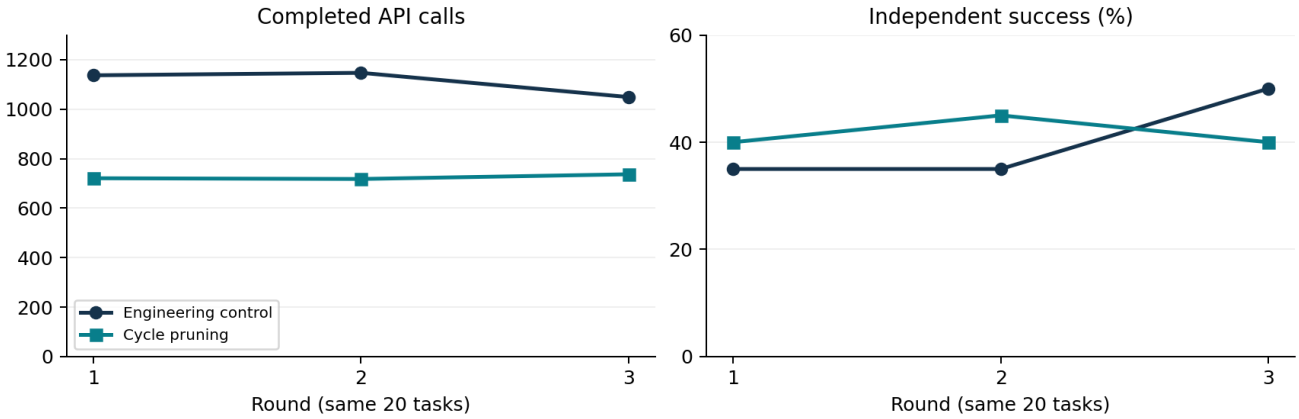


Figure 1. Calls decline in every round; accuracy differences change sign. Lines connect repeated runs of the same tasks, not new task samples.

Across three rounds	Control	Cycle pruning
Success mean +/- round SD	40.0% +/- 8.66 pp	41.67% +/- 2.89 pp
Calls mean +/- round SD	1111.00 +/- 53.93	725.33 +/- 10.21
Pooled successful task-runs	24/60	25/60
Input + output tokens	3,613,403	2,387,485
Estimated fees, CNY	1.00033-2.00067	0.65889-1.31778

Total calls fall from 3,333 to 2,176 (-34.7%), returned tokens by 33.9%, and estimated fees by 34.1%. Across paired observations there are 18 joint successes, 29 joint failures, 7 gains and 6 losses. The mean success difference is only +1.67 percentage points; round differences are +5, +10 and -10 points (sample SD 10.41 points).

The smaller pruned-arm round-level SD is not evidence of more reliable task-wise outcomes: both arms have 6 tasks successful in all rounds, 9 failing in all rounds and 5 mixed. The defensible result is repeated resource reduction on this subset. No stable accuracy gain or statistical significance is established.

5. Failure analysis and debug diary

Round-three losses

Round three has three control-success/pruned-failure cases and one gain, producing a net loss of two. Offline replay of all 59 modeled nodes in the six loss-case trees (including roots) finds no illegal position transition. All eight recorded cuts are genuine ancestor revisits; none is a complete goal. Each logged control solution is cycle-free.

Task	Unexplored continuation at the shared prefix	Observed failure
instance-36	At depth 4, pick up orange; then stack it on blue	Only unstack-blue-from-red was generated (score 0); the correct action is absent at node 5.
instance-69	At depth 4, pick up red; then stack it on yellow	Only unstack-blue-from-orange was generated at node 8; three attempts terminate at cycles.
instance-73	At depth 4, pick up red; then stack it on yellow	Only unstack-blue-from-orange was generated at node 6; no alternative is compared.

For 36 and 73, the same policy prompt hash and temperature yield different actions before the first cut. For 69 the same-prompt divergence occurs after an earlier cut, so it is not an untreated random comparison. The logs do not show direct pruning of a valid solution; they show independently sampled paths, one-candidate rollouts and inadequate repair opportunities within three attempts.

Positive backup from a no-progress loop

In instance-69, picking orange up and putting it back produces a loop yet leaves the orange root branch with Q=0.25, versus 0.15 for the useful red branch. Before the last attempt, both have one visit and the root has two. With exploration weight 1, UCT is 1.08255 versus 0.98255, so search returns to orange. Closing a leaf alone does not define a failure value for the rollout. This motivates cycle-aware backup as a separate future ablation, not a fix silently added to the reported treatment.

Main blockers and resolutions

Blocker	Resolution and scope
Large optional dependency tree	Pinned an API-only environment; unused backends are outside validation scope.
Repeated actions / exhausted-leaf stalling	Shared engineering fixes form the explicit control. They are not attributed to the isolated cycle treatment.
Misleading model-generated terminal state	Replay all selected actions independently; depth-only leaves do not count as successes.
Budget, hidden thinking and billing uncertainty	Disable thinking and SDK retries; cap output; retain starts, usage and failed requests; report fee intervals.
macOS sandbox OpenMP shared-memory failure	Run the same network-disabled evaluation in a normal local process; no paid rerun.

6. Exploratory low-score widening pilot

A later, independent extension tests one proposed response to the candidate bottleneck. It inherits cycle pruning and changes only simulation expansion: if a node has exactly one total candidate, which is open and scored at or below zero, and more legal untried actions exist, add one candidate. Unscored sentinel -1 is excluded. Each node may trigger once and each task twice. The original candidate is retained; the original argmax, cycle rule and backup are unchanged.

Admission requires four unused request slots and 10,000 returned-token headroom, conservatively allowing one policy call plus up to three PRM attempts. This is a precheck, not a proof of future token use. The existing per-task guards remain active. Ledger errors, missing usage or unaccounted requests deny widening. Nine network-disabled regression groups and four real-CLI dry runs passed before the paid pilot.

Development result	Original cycle arm	Cycle + low-score widening
instance-11	Failure; 60 requests	Failure; 42 requests
instance-13	Success; 26 requests	Success; 26 requests
Independent successes	1/2	1/2
Total requests / returned tokens	86 / 93,488	68 / 74,228
Estimated CNY fees	0.025769-0.051537	0.020601-0.041203

The mechanism fires twice in instance-11. It adds put-down-blue and pick-up-orange, both scored zero; both remain unmodeled with visit count zero in the final tree. Direct generation/scoring overhead is five requests and 6,082 tokens. Instance-13 does not trigger widening. Equal scores leave the original candidate first under the retained argmax rule.

Negative finding: implementation and budget gating work, but no success gain is observed and the added actions are not explored in this pilot. The total 18-request difference cannot be attributed to beneficial use of added candidates: independent sampling and later path changes remain confounds. Direct widening overhead and whole-run cost differences are distinct quantities.

What would be tested next?

Future versions could separately test tie-aware exploration, bounded repair at the decision that led to a cycle, and a cycle-specific failure contribution to backup. Necessary low-scored actions must not be removed wholesale. The current report does not combine these changes or claim that cached-log counterfactuals predict unseen model responses. Diagnosis has now used the original test cases; future tuning on them would turn that set into a regression set, requiring a fresh holdout for generalization claims.

This pilot contains only two familiar development tasks and one run per arm. No test task or additional paid retry was used for the new extension.

7. Conclusions, limitations and artifacts

Conclusions supported by the evidence

Workflow reproduction: the pinned repository can produce legal multi-step BlocksWorld plans with a replacement model, preserve run artifacts and support external search subclasses. **Primary modification:** ancestor-cycle pruning lowers calls in all three repetitions on this subset; its stopping behavior uses less realized search. **Accuracy:** the direction of the paired success effect is inconsistent. **Exploratory widening:** the bounded mechanism activates but does not improve development success in this pilot.

Threats to validity

The model/provider, sample selection and iteration count differ from the paper. The common engineering fixes prevent an untouched-author-baseline interpretation. Twenty consecutive tasks and three repetitions limit generalization; observations are clustered by task. Local seeds do not control remote sampling. Provider model versions, cache state and concurrent scheduling may change outcomes and fees. Canonicalization is based on model text and conservatively skips uncertain parses. Fee estimates exclude provider-bill verification. A clean-machine dependency rebuild was not performed; installation and verification are documented separately. Source/configuration hashes preserve the evaluated state but are not a substitute for an immutable provider model snapshot.

Reproducing the evidence without new model calls

```
cd <repository-root>
# Install the pinned runtime and upstream source (network, no model API).
PYTHON_BIN=python3.12 bash submission_setup.sh
.venv/bin/python scripts/cycle_repeat_report.py
.venv/bin/python scripts/analyze_cycle_losses.py
.venv/bin/python scripts/low_score_report.py results/low_score_dev_20261104
```

The accompanying GitHub-ready package includes the selected real request ledgers, trees, terminal outputs, invocation metadata and protocols required by these commands. Read its README for credentials, platform caveats and explicit paid-run switches. The .env file, virtual environment and upstream nested Git repository are excluded. The original upstream source is fetched at the pinned SHA. The provided setup recipe is not a claim of a fresh-machine validation.

References and primary evidence

[1] X. Li and Y. Tao. LiTS: A Modular Framework for LLM Tree Search. arXiv:2603.00631v2, 17 May 2026. Section 3.1 and Table 3. arxiv.org/html/2603.00631v2.

[2] Author repository, pinned commit 1ef347bee7b3f2c10dc504342589b8c951794c2d (7 October 2026). github.com/xinzhel/lits-llm.

[3] This project: reports/cycle_protocol_20261104.json; cycle_stability_20261104.json; cycle_round3_loss_analysis_20261104.json; low_score_dev_20261104.json; and the matching results/ directories. Measurements: 23 October and 4 November 2026; report prepared 5 November and revised 9 November 2026.

[4] DeepSeek API documentation, pricing snapshot checked 4 November 2026. api-docs.deepseek.com/zh-cn/quick_start/pricing/. Local fee code: scripts/deepseek_cost.py.